A Real Time Guitar Amp Simulation In A Hardware Implementation

Hardware Design By	Garin Anggara
Firmware Design By	Garin Anggara
Time Stamps	December 20th 2024
Version	1.0

Objective

The goal of the project is to create a real time guitar amp simulation software. Real time hardware implementation is achieved through the microcontroller and external codec, which accepts real time guitar signals as an input. Microcontroller will process the input signal acquired by codec, with some filtering and nonlinearity to introduce distortion. Simulation does not necessarily mimic the actual guitar amplifier to the point of view of math. If the resulting audio output is decent enough with tolerable amount of latency, then the goal is achieved.

Hardware Specification

Dimension (mm)	128 x 116
Microcontroller	STM32H743ZIT6
Main CPU Clock	480 MHz
Dedicated Codec	TLV320AIC3104
Sample Rate	48000 Hz (actual is 47753 Hz)
Audio Bit Depth	24-bit
Audio IO	RST Mono Jack 6.35 (1 In, 2 Out)

Hardware System Diagram

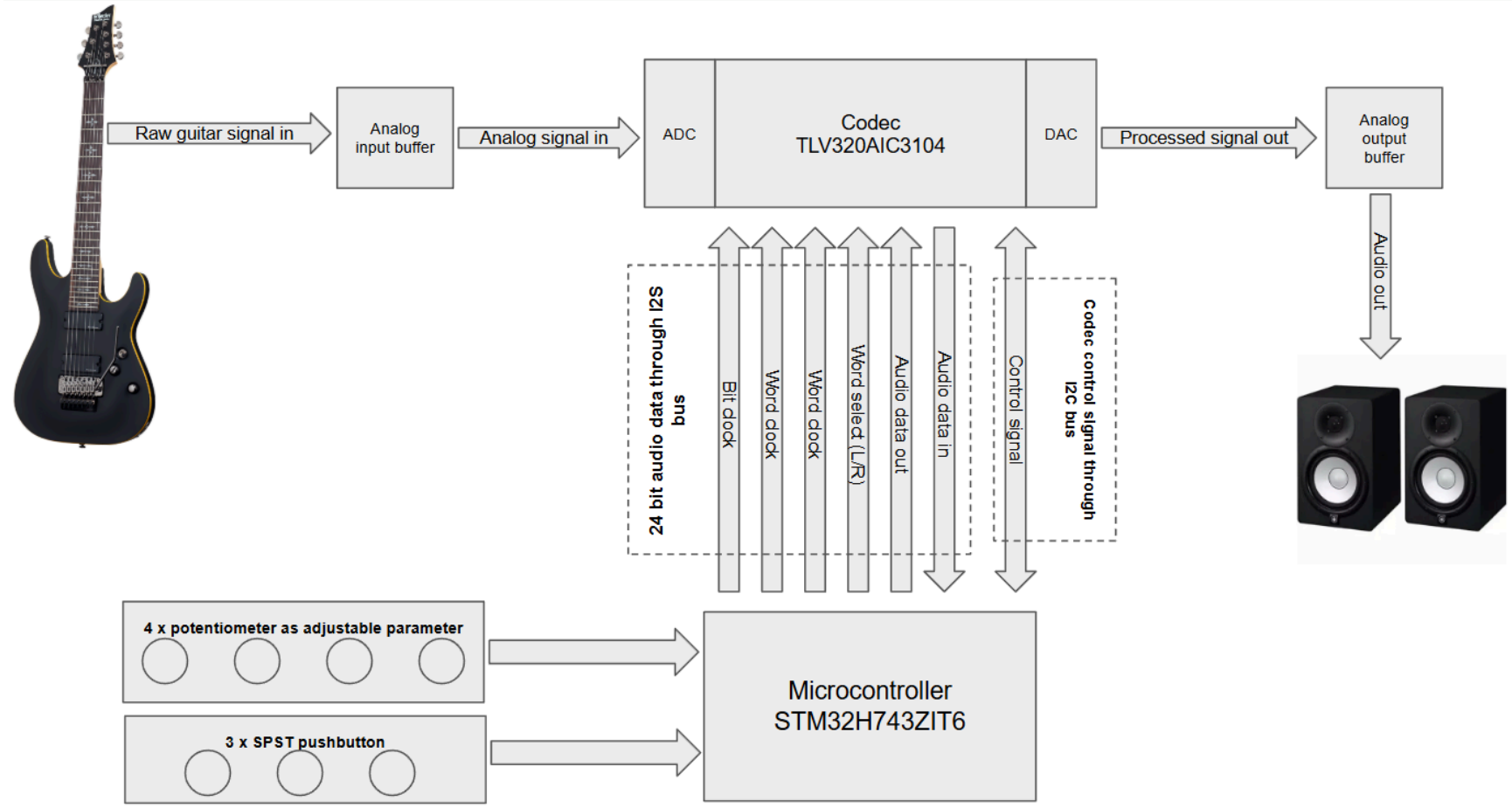

Figure 1. Hardware System Diagram

Analog raw guitar signal is received by analog input buffer. The analog input buffer consists of an op-amp with voltage follower configuration. It provides analog high pass and low pass filtering, also buffers the signal so it does not get attenuated.

Output of the analog input buffer is then converted into a digital signal by ADC part of the codec. Then the codec will send the 24 bit data to the microcontroller with a word clock of nearly 48 kHz. This word clock is what we call sample rate.

Notice there are two different communication lines between codec and microcontroller. While the audio data is transferred through I2S bus, control signal is transferred through I2C bus. All those parameters like bit depth, sample rate, ADC mode, DAC mode, etc. are mandatory to be configured. In coding practice, we need to write a driver for the codec in the first place.

The audio data sent by codec via I2S bus is received by the microcontroller. Then the microcontroller will perform several processes like filtering, nonlinearity, etc. Once the audio processing is completed, then the microcontroller will send the processed audio data back into the codec. Now the DAC will perform digital to analog conversion for it to be sent into the analog output buffer.

Four potentiometers are utilized for adjustable parameters such as gain, volume, bass, and treble. They are connected to the internal ADC input of the microcontroller.

There are three available push buttons. But only two are utilized. The first button is to switch between bypass and active mode. Active mode is when the amplifier is on, the audio output will provide distorted heavy guitar sound. While the bypass mode is when the amplifier is off, the audio output will provide clean unprocessed guitar sound.

The second button is to switch between low gain and high gain mode. This button impact is only visible on active mode.

Guitar Amp Sim Anatomy

Amp Sim Anatomy here meaning the order of signal processing order as shown in figure 2. It is the main process where the raw guitar sound is processed into desired heavy metal guitar sound.

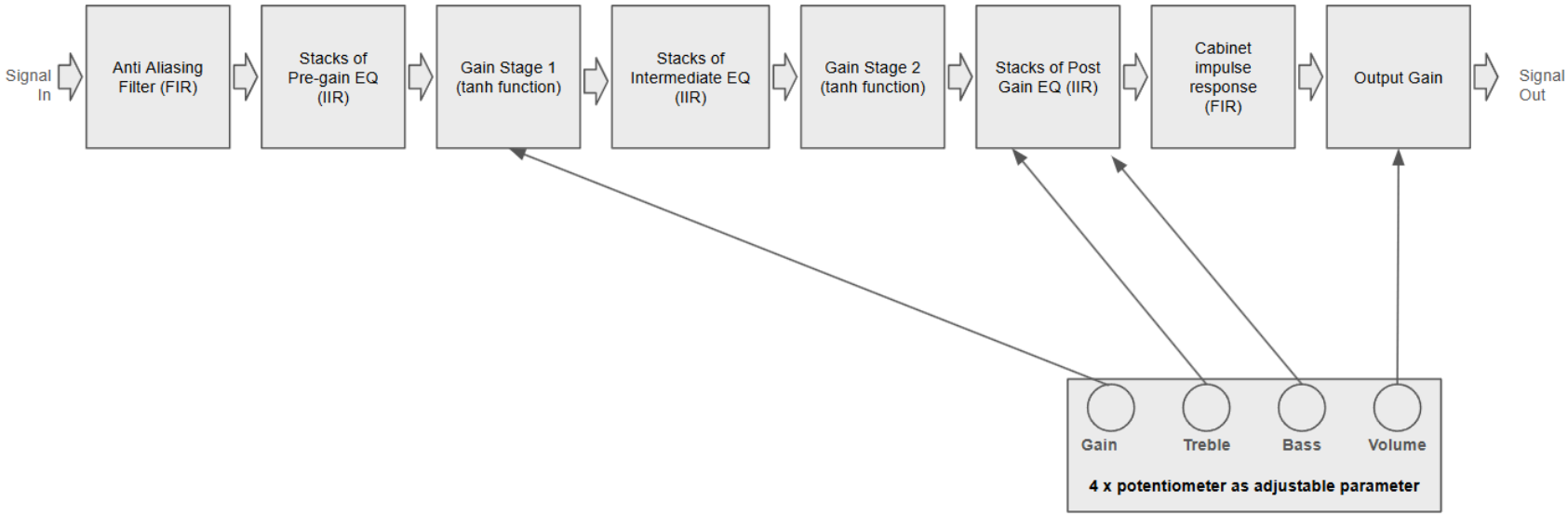

Figure 2. Order of Signal Processing

There are three mathematical ingredients that is used for sound processing :

1. Finite impulse response filter, for anti aliasing filter and cabinet impulse response
2. Infinite impulse response filter, for adjustable equalizer in several stages
3. Gain stage with nonlinearity, specifically a function that close to tangent hyperbolic function, is used to introduce harmonic distortion in order to create distorted heavy guitar sound

Anti aliasing filter is applied in the first part of the block. It is low pass filter in the form of FIR filter with cut off frequency at 10 kHz as shown in figure 3. The length of the filter coefficient is 47, occupying memory size of 188 byte float (47 x 4 byte/number).

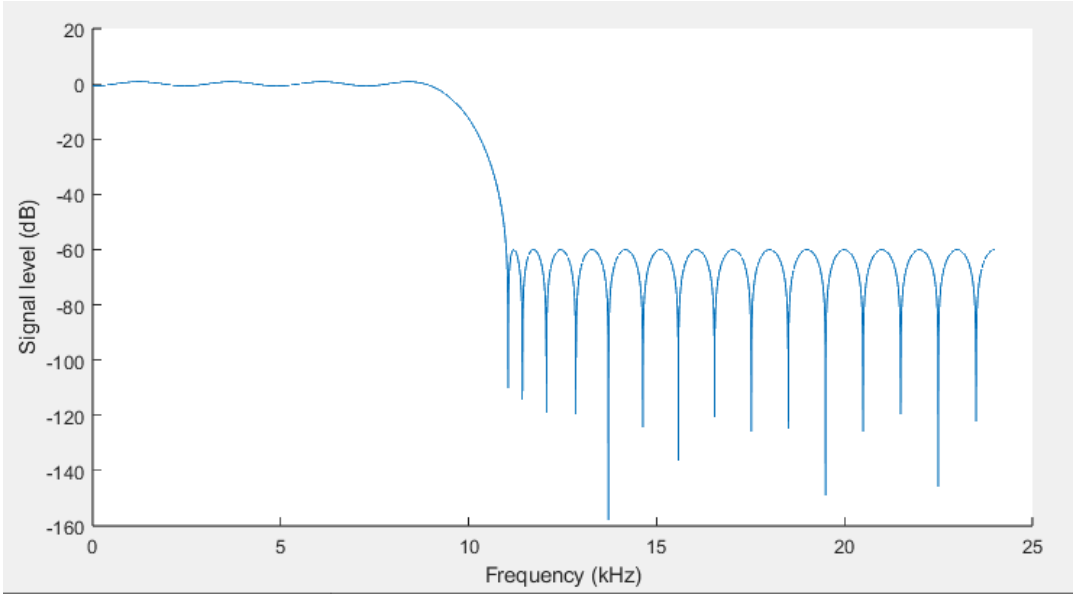

Figure 3. Anti Aliasing Filter Frequency Response

The second block is the pre-gain EQ, a stack of several equalizers in the form of an infinite impulse response filter. This pre-gain EQ is necessary to shape the tone before entering the first gain stage. Also if low frequency is fed into the nonlinear gain stage, the resulting sound is quite inconvenient. Hence why, high pass filter is mandatory in the pre-gain EQ. Each of the pre-gain EQ components are detailed in table 1.

Table 1. Pre-Gain Equalizers

Frequency (Hz)	Q	Gain factor	Filter type
150	0.74	1.51	Bell
623	0.6	1	Highpass
680	0.94	1.4	Bell
1500	0.51	2.31	Bell

Figure 4 shows the EQ curve of the pre-gain stage EQ. It provides boost to the middle frequencies from around 680 Hz to 1500 Hz.

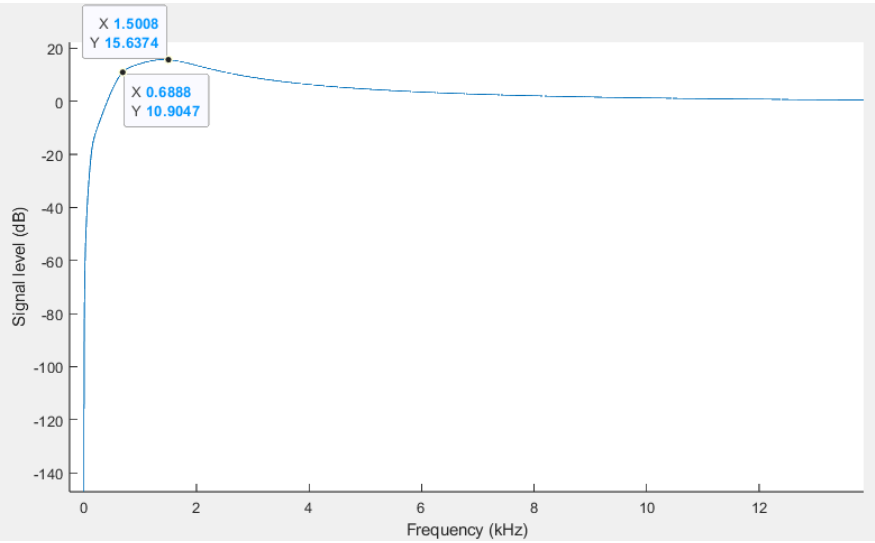

Figure 4. Frequency Response of Pre-Gain EQ

The third block is the gain stage 1. A linear gain stage with linear relationship between input and output is implemented with one multiplication factor (gain factor). For example, a gain factor of 0.5 will attenuate a signal into half of the input. On the other hand, nonlinear gain is not a single multiplier number. In this project, I use a made up function that shapes close to tangent hyperbolic function to introduce nonlinearity. The slope and the saturation point for the positive signals are different from the negative signal. In other words, it is called asymmetric clipping. Nonlinearity profile of gain stage 1 can be seen in figure 5.

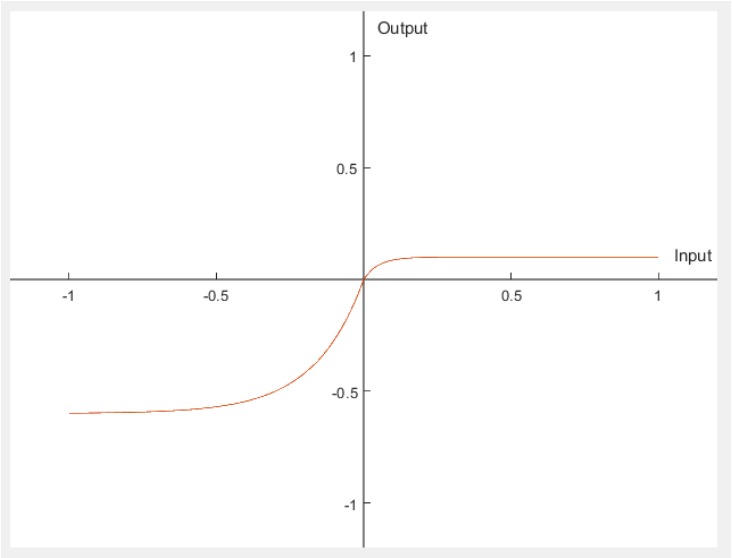

Figure 5. Gain Stage 1 Nonlinearity

Most guitar amplifiers contain a thermionic valve as an amplification component. In heavy guitar music such as in rock music, most likely the guitar amp is configured in high gain. Due to the characteristic of valve amplification (also depending on the bias configuration), the

signal will be clipped asymmetricaly when driven by a high level of input signal. And hence why, gain stage 1, and later also gain stage 2, were designed in asymmetric shape.

For the intermediate EQ stacks, although it is shown inside the block in figure 2, it is optional. The function to execute the IIR filter for intermediate EQ is there, but the parameter is disabled. The block is there for further development purposes in case different tone characteristics are required.

As have mentioned before, the gain stage 2 is also asymmetrical. Difference is that the saturation point of the gain stage 2 is slightly higher than the gain stage 1 for both positive and negative input signals. Together the gain stage 1 and gain stage 2 provides high gain sounds for the guitar. Nonlinearity profile of gain stage 2 can be seen in figure 6.

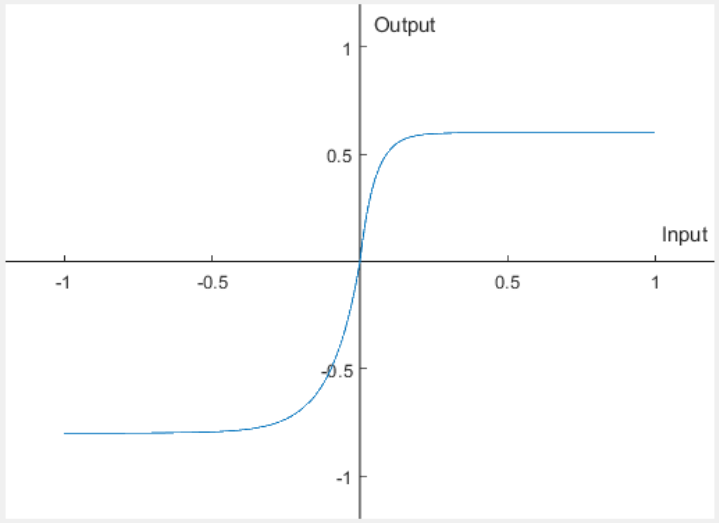

Figure 6. Gain Stage 2 Nonlinearity

One of the potentiometers is attached to the input level of gain stage 2. This provides the ability for the user to adjust the gain on the go, by turning the knob.

After all the gain stage, the super distorted signal is now being filtered again by post-gain EQ stacks. Figure 4 shows the EQ curve of the pre-gain stage EQ. Each of the post-gain EQ components are detailed in table 2.

Table 2. Post-Gain Equalizers

Frequency (Hz)	Q	Gain factor	Filter type
80	0.7	1	Highpass
90	0.96	1.652	Bell
2300	0.71	0.673	Bell
4000	0.96	1.4	Bell
12000	0.66	1	Lowpass

For 80 Hz and 12000 Hz, they are attached to two of the potentiometers. One for treble control, and the other one is for bass control. Adjustable bass and treble are achieved by moving the cut off frequency of the highpass and lowpass filter. The other EQ frequencies are not adjustable on the go, and must be tuned by changing the code. Post-gain EQ curve can be seen in figure 7.

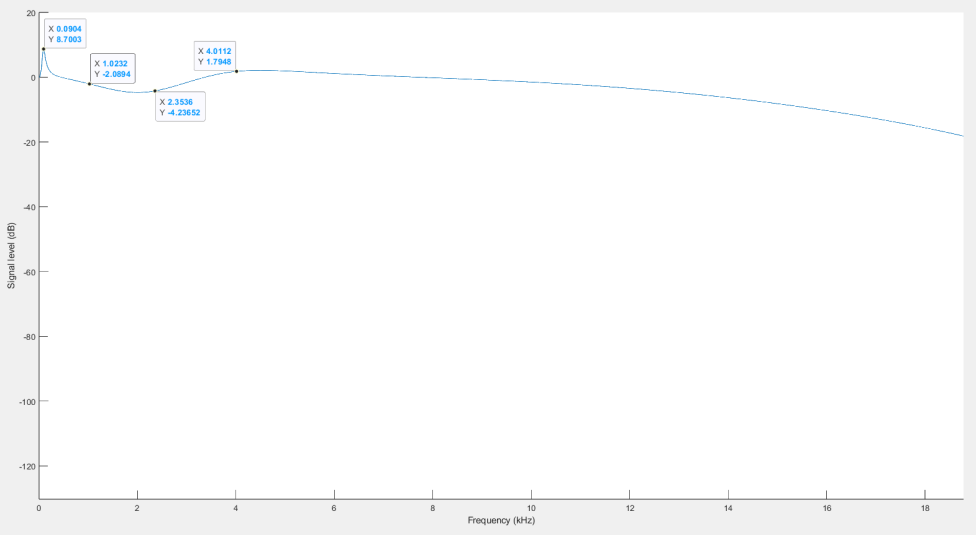

Figure 7. Frequency Response of Pre-Gain EQ

Finally the last part of the block is the impulse response of the speaker cabinet. In guitar terminology, we can call it cab IR. The cab IR that is used in this project is a Messa Boogie 4x12 speaker cabinet, captured by SM-57 microphone. This IR cab is what emulates the guitar amp speaker. Without it, the sound will be so harsh and inconvenient. TLength of the IR cab is 1024, occupying 4 MB of program memory. IR cab curve of Messa Boogie speaker cabinet can be seen in figure 8.

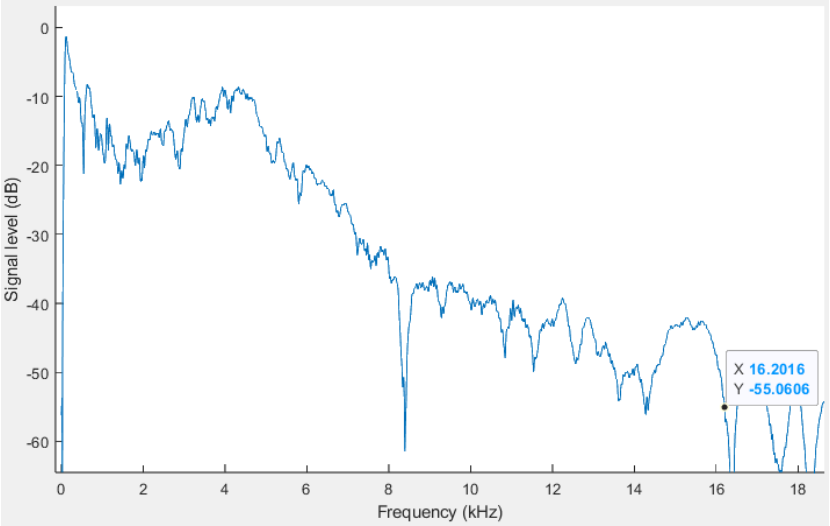

Figure 8. Cab IR Frequency Response

Software Implementation

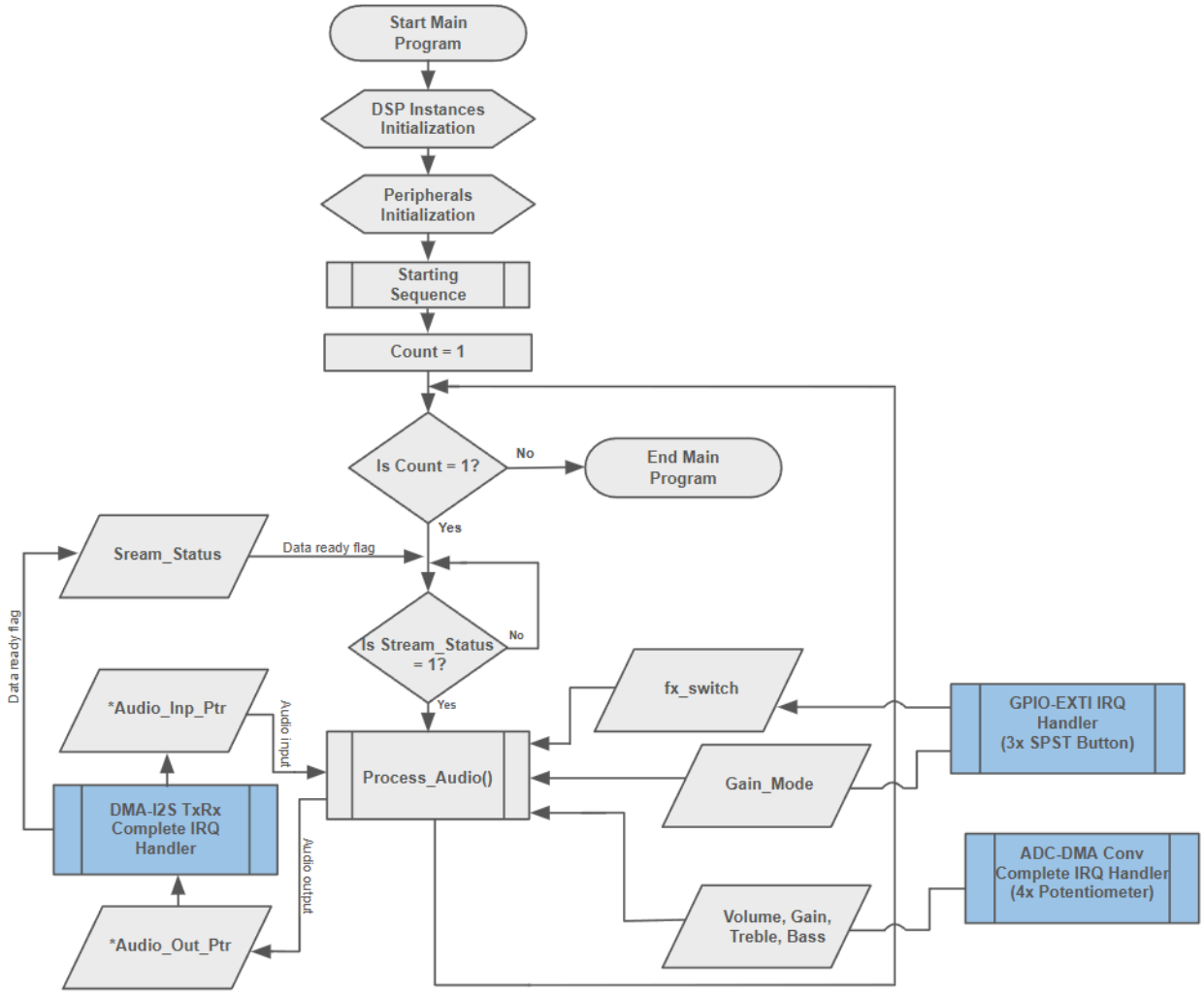

Figure 9. Main Program Flow Chart

Figure 9 is the flowchart that pictures a simplified version of the main program that is running inside the microcontroller. It consists of several peripherals and DSP instances initialization, one infinite loop, three interrupt handlers from three peripheral service routines, data conversion routine, and the DSP processing routine itself.

1. DSP instances initialization

   There are three types of DSP instances : IIR filter instance, gain stage instance, and FIR filter instance. Among those three, only three are adjustable. The first is the gain input of the gain stage 1. Second is the cutoff frequency of the high pass IIR filter. And the last is the cutoff frequency of the low pass IIR filter.

2. Peripherals initialization

   There are several peripherals that needs to be initialized :

- System clock configuration
  Main clock (CPU clock) is configured at 480 MHz
- Peripheral clock configuration
- GPIO initialization
- DMA initialization
- I2S initialization
- ADC initialization
- Timer initialization

3. Starting Sequence
   - Internal ADC calibration
   - Internal ADC start conversion
   - Timer start
   - Codec start
     Microcontroller will send a command to the codec via I2C bus to start the audio ADC & audio DAC
   - I2S and DMA transmit & receive start

4. Main infinite loop

   Main loop is always checking for the *Stream_Status* variable. Stream status is a data ready flag. When *Stream_Status contains a value* of 1, then the program will perform data processing. The data processing function is named *Process_Audio()*.

   The *Stream_Status* variable is updated with the value of 1 by DMA-I2S IRQ handler (*HAL_I2SEx_TxRxHalfCpltCallback* and *HAL_I2SEx_TxRxCpltCallback*). Those two handlers are called only when DMA issues an interrupt request for I2S-DMA transmission complete and half complete. So the *Process_Audio()* is only called when there is new data available in the *Audio_ADC_Buff.*

   The data inside the *Audio_ADC_Buff* array is accessed by *Process_Audio()* via pointer named **Audio_Inp_Ptr*. When the half transmission IRQ is issued, then **Audio_Inp_Ptr* is pointed to the index 0 of the array (*Audio_ADC_Buff[0])*. When the full transmission IRQ is issued, then **Audio_Inp_Ptr* is pointed to the middle of the array (*Audio_ADC_Buff[Audio_Buffer_Size/2]).*

   Process_Audio() function contains two important parts. The first one is data alignment and conversion (both for input and output). The second one is the DSP itself. It also contains a finite loop based on *Audio_Buffer_Size* to process all the data inside *Audio_ADC_Buff*.

   The more accurate flowchart of the *Process_Audio()* is shown in figure 10.

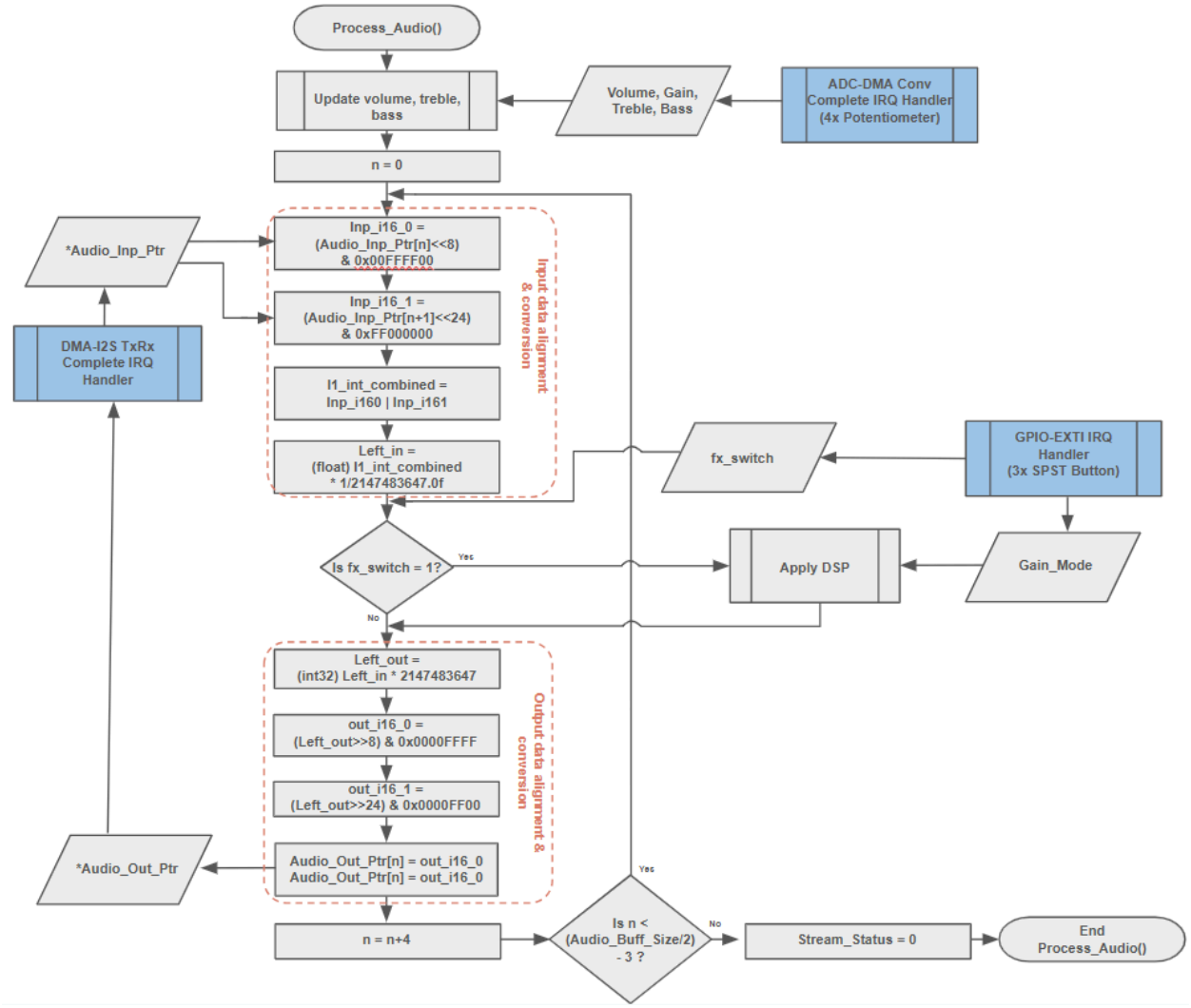

Figure 10. Flowchart of *Audio_Process()* function

All adjustable parameters are updated first. Updated values are sent by ADC-DMA Conv Complete IRQ handler (*HAL_ADC_ConvCpltCallback*) once internal ADC conversion is completed. Then the program loops to process all input data in the buffer from n = 0 until n >= (*Audio_Buff_Size/2) - 3* via **Audio_inp_Ptr* pointer.

One pack of data contains two audio channels, left channel and right channel. One data point of the left channel occupies two rows of array, because the transmission is in a 32-bit data frame, but the DMA buffer only can contain 16-bit data. Hence 4 rows of the array are occupied for one pack of data (2 rows for the left channel, and 2 rows for the right channel). Even though we will only process left channel data only, the right channel will still be transmitted by the codec. The right channel will only contain two rows of an array with a value of zeroes.

Since one data point is contained in two rows of an array, we must combine two 16-bit data into one 32-bit data. The byte order is important here as the endian is becoming a concern. Fortunately we can find some recommendations in the STM32

reference manual. As we see in figure 11, the STM32 reference manual recommends us to follow such byte order for memory width of 16 bit, and peripheral width of 32 bit.

Table 89. Packing/unpacking and endian behavior (bit PINC = MINC = 1)

AHB memory port width	AHB peripheral port width	Number of data items to transfer (NDT)	Memory transfer number	Memory port address / byte lane	Peripheral transfer number	Peripheral port address / byte lane	
						PINCOS = 1	PINCOS = 0
8	8	4	1 2 3 4	0x0 / B0[7:0] 0x1 / B1[7:0] 0x2 / B2[7:0] 0x3 / B3[7:0]	1 2 3 4	0x0 / B0[7:0] 0x4 / B1[7:0] 0x8 / B2[7:0] 0xC / B3[7:0]	0x0 / B0[7:0] 0x1 / B1[7:0] 0x2 / B2[7:0] 0x3 / B3[7:0]
8	16	2	1 2 3 4	0x0 / B0[7:0] 0x1 / B1[7:0] 0x2 / B2[7:0] 0x3 / B3[7:0]	1 2	0x0 / B1|B0[15:0] 0x4 / B3|B2[15:0]	0x0 / B1|B0[15:0] 0x2 / B3|B2[15:0]
8	32	1	1 2 3 4	0x0 / B0[7:0] 0x1 / B1[7:0] 0x2 / B2[7:0] 0x3 / B3[7:0]	1	0x0 / B3|B2|B1|B0[31:0]	0x0 / B3|B2|B1|B0[31:0]
16	8	4	1 2	0x0 / B1|B0[15:0] 0x2 / B3|B2[15:0]	1 2 3 4	0x0 / B0[7:0] 0x4 / B1[7:0] 0x8 / B2[7:0] 0xC / B3[7:0]	0x0 / B0[7:0] 0x1 / B1[7:0] 0x2 / B2[7:0] 0x3 / B3[7:0]
16	16	2	1 2	0x0 / B1|B0[15:0] 0x2 / B1|B0[15:0]	1 2	0x0 / B1|B0[15:0] 0x4 / B3|B2[15:0]	0x0 / B1|B0[15:0] 0x2 / B3|B2[15:0]
16	32	1	1 2	0x0 / B1|B0[15:0] 0x2 / B3|B2[15:0]	1	0x0 / B3|B2|B1|B0[31:0]	0x0 / B3|B2|B1|B0[31:0]

Figure 11. DMA Data Packing/Unpacking and Endian Behavior

For such a reason, data alignment is required for the input and output parts. After the byte combine and alignment, the data is becoming a 32-bit signed integer. We need to convert a 32-bit signed integer into a 32-bit float. The actual resolution of the audio data is actually 24-bit. But it is converted into a 32-bit scale instead, and then converted again into a 32-bit float by conversion factor of 1/2147483647,0.

After all the data preparation process, then the 32-bit float audio data will be processed by the “Apply DSP” process. The order of DSP is the same as figure 2.

For the FIR filter block, convolution sum is used to implement FIR filter. The equation 1 is the convolution sum equation.

$$f[x] * g[x] = \sum_{k=-\infty}^{\infty} f[k] \cdot g[x-k]$$

Equation 1. Convolution Sum

For the code implementation, equation 1 is then translated into a codes within *Process_FIR_filter()* function :

```
float Process_FIR_Filter(FIR_Struct *FIRobj, float Inp){

    static uint16_t Convo_Idx = 0;
    static float Out = 0.0f;

       Out = 0.0f;
       if (FIRobj->Switch == FIR_ON){
              FIRobj->pBuffers[FIRobj->Buf_Idx] = Inp;
              FIRobj->Buf_Idx++;
              if (FIRobj->Buf_Idx == FIRobj->Coef_Size){
                     FIRobj->Buf_Idx = 0;
              }
              Convo_Idx = FIRobj->Buf_Idx;
              for (uint16_t n = 0; n < FIRobj->Coef_Size; n++){
                     if (Convo_Idx> 0){
                            Convo_Idx--;
                     }
                     else{
                            Convo_Idx = FIRobj->Coef_Size - 1;
                     }
                     Out = Out + (FIRobj->pCoefs[n] * FIRobj->pBuffers[Convo_Idx]);
                     //Out = FIRobj->Lvl_Out;
              }
       }
       else{
              Out = Inp;
       }

       return Out;
}
```

For nonlinearity, I made up a function that looks close to tangent hyperbolic. There are two different functions for positive signals, and the other one is for negative signals. The following code shows the implementation of the nonlinearity by *Proces_Drive()* function.

```
float Process_Drive(Drive_Struct *Drvobj, float Inp){
       float Out = 0.0f;
       float Scaled_Inp = 0.0f;

       if (Drvobj->Switch == DRIVE_ON){
              Scaled_Inp = Drvobj->Inp_Lvl * Inp;
              if (Inp >= 0.0f){
                     Out =    Drvobj->Pos_Lim - (Drvobj->Pos_Lim*
exp(-1.0f*Drvobj->Pos_Slope
                                  * Scaled_Inp));
              }
              else{
                     Out = -1.0f*Drvobj->Pos_Lim + (Drvobj->Pos_Lim* exp(Drvobj->Neg_Slope *
                            Scaled_Inp));
              }
              Out = Out * Drvobj->Out_Lvl;
       }
       else{
              Out = Inp;
       }

       return Out;
}
```

For IIR filter block, it uses biquad IIR filter. To implement a biquad IIR filter, we must apply equation 2.

```
y[n] = (b0/a0)*x[n] + (b1/a0)*x[n-1] + (b2/a0)*x[n-2]
                    - (a1/a0)*y[n-1] - (a2/a0)*y[n-2]
```

Equation 2. Biquad IIR filter in a digital time domain

Equation 2 then translated into a following code :

```
float Process_QIIR_Filter(QIIR_Struct *QIIRobj, float Inp){
	float Out = 0.0f;

	if (QIIRobj->Switch == QIIR_ON){
		QIIRobj->Inp_Buff[2] = QIIRobj->Inp_Buff[1];
		QIIRobj->Inp_Buff[1] = QIIRobj->Inp_Buff[0];
		QIIRobj->Inp_Buff[0] = Inp;

		QIIRobj->Out_Buff[2] = QIIRobj->Out_Buff[1];
		QIIRobj->Out_Buff[1] = QIIRobj->Out_Buff[0];
		QIIRobj->Out_Buff[0] = QIIRobj->a[0] * ((QIIRobj->b[0]*
				       QIIRobj->Inp_Buff[0]) +
				       (QIIRobj->b[1] * QIIRobj->Inp_Buff[1]) +
				      (QIIRobj->b[2] * QIIRobj->Inp_Buff[2]) -
				      (QIIRobj->a[1] * QIIRobj->Out_Buff[1]) -
				      (QIIRobj->a[2] * QIIRobj->Out_Buff[2]) );
		Out = QIIRobj->Out_Buff[0];
	}
	else{
		Out = Inp;
	}

	return Out;
}
```

With “y” is the output signal at “n”, and “x” is the input signal at “n”. The parameters “a” and “b” need to be computed first, and they depend on the type of filters. There are 3 type of IIR filters that is used in this project :

- High pass filter
- Low pass filter
- Bell filter

Table 3 shows how to compute parameters “a” and “b” for biquad filter. There are 8 type, but in this project only 3 of them are used.

Table 3. Biquad IIR Filter Pre-computed Parameters

LPF: H(s) = 1 / (s^2 + s/Q + 1) b0 = (1 - cos(w0))/2 b1 = 1 - cos(w0) b2 = (1 - cos(w0))/2 a0 = 1 + alpha a1 = -2*cos(w0) a2 = 1 - alpha	APF: H(s) = (s^2 - s/Q + 1) / (s^2 + s/Q + 1) b0 = 1 - alpha b1 = -2*cos(w0) b2 = 1 + alpha a0 = 1 + alpha a1 = -2*cos(w0) a2 = 1 - alpha
HPF: H(s) = s^2 / (s^2 + s/Q + 1) b0 = (1 + cos(w0))/2 b1 = -(1 + cos(w0)) b2 = (1 + cos(w0))/2 a0 = 1 + alpha a1 = -2*cos(w0) a2 = 1 - alpha	peakingEQ: H(s) = (s^2 + s*(A/Q) + 1) / (s^2 + s/(A*Q) + 1) b0 = 1 + alpha*A b1 = -2*cos(w0) b2 = 1 - alpha*A a0 = 1 + alpha/A a1 = -2*cos(w0) a2 = 1 - alpha/A
BPF: H(s) = (s/Q) / (s^2 + s/Q + 1) b0 = alpha b1 = 0 b2 = -alpha a0 = 1 + alpha a1 = -2*cos(w0) a2 = 1 - alpha	lowShelf: H(s) = A * (s^2 + (sqrt(A)/Q)*s + A)/(A*s^2 + (sqrt(A)/Q)*s + 1) b0 = A*((A+1) - (A-1)*cos(w0) + 2*sqrt(A)*alpha) b1 = 2*A*((A-1) - (A+1)*cos(w0)) b2 = A*((A+1) - (A-1)*cos(w0) - 2*sqrt(A)*alpha) a0 = (A+1) + (A-1)*cos(w0) + 2*sqrt(A)*alpha a1 = -2*((A-1) + (A+1)*cos(w0)) a2 = (A+1) + (A-1)*cos(w0) - 2*sqrt(A)*alpha
notch: H(s) = (s^2 + 1) / (s^2 + s/Q + 1) b0 = 1 b1 = -2*cos(w0) b2 = 1 a0 = 1 + alpha a1 = -2*cos(w0) a2 = 1 - alpha	highShelf: H(s) = A * (A*s^2 + (sqrt(A)/Q)*s + 1)/(s^2 + (sqrt(A)/Q)*s + A) b0 = A*((A+1) + (A-1)*cos(w0) + 2*sqrt(A)*alpha) b1 = -2*A*((A-1) + (A+1)*cos(w0)) b2 = A*((A+1) + (A-1)*cos(w0) - 2*sqrt(A)*alpha) a0 = (A+1) - (A-1)*cos(w0) + 2*sqrt(A)*alpha a1 = 2*((A-1) - (A+1)*cos(w0)) a2 = (A+1) - (A-1)*cos(w0) - 2*sqrt(A)*alpha

After knowing how to compute “a” and “b”, then we can substitute the number from table 1 and table 2 into the equation in table 3 depending on the filter type.